Competitive Programming Notes

Everything under `notes/`, collected into one document.

Contents

Number Theory Knowledge

Application in Euler's theorem

if a and m are relatively prime.

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

In the particular case when m is prime, Euler's theorem turns into **Fermat's little theorem**:

$$a^{m-1} \equiv 1 \pmod{m}$$

As immediate consequence we also get the equivalence:

$$a^n \equiv a^{n \bmod \phi(m)} \pmod{m}$$

This allows computing $x^n \bmod m$ efficiently for **not coprime** x and m. For arbitrary x, m and $n \geq \log_2 m$:

$$x^n \equiv x^{\phi(m)+[n \bmod \phi(m)]} \bmod m$$

> NOTE
>
> Problem $(2^{2^n}+1) \bmod k$
>
> Suppose pow = $(\phi(m) + [n \bmod \phi(m)]) \bmod \phi(m)$
>
> if (pow == 0) pow = $\phi(m)$

Why Handle pow = 0?

In your code, you compute:

$$\text{pow} = 2^n \bmod \phi(k)$$

However, when 2^n is a multiple of $\phi(k)$, the result of $2^n \bmod \phi(k)$ will be 0. But this creates an issue when using it in modular exponentiation:

$$2^{\text{pow}} \bmod k$$

If $\text{pow} = 0$, it means you're computing:

$$2^0 \bmod k$$

which is always 1. But mathematically, when n is large, we expect:

$$2^n \equiv 2^{\phi(k)+(n \bmod \phi(k))} \bmod k$$

By Euler's theorem:

$$x^{\phi(m)} \equiv 1 \bmod m$$

so we need to adjust by replacing 0 with $\phi(k)$ because:

$$2^{\phi(k)} \equiv 1 \mod k$$

Thus, when $\text{pow} = 0$, setting it to $\phi(k)$ ensures the exponentiation still produces a meaningful result.

Modular Inverse

- For an arbitrary (but coprime) modulus m: $a^{\phi(m)-1} \equiv a^{-1} \mod m$
- For a prime modulus m: $a^{m-2} \equiv a^{-1} \mod m$
- if a and m are coprime, then $a^{-1} \mod m$ can be found using the extended Euclidean algorithm.

Computing $F(n)$ Modulo a Composite Number

Notes

1. We want to compute: $F(n) \bmod C$ where C is **not** prime.

1. **Why Factor C?**
   * When C is prime, we can use straightforward tools (like Fermat's Little Theorem) because $\mathbb{Z}/p\mathbb{Z}$ is a field.
   * For a composite C, $\mathbb{Z}/C\mathbb{Z}$ is **not** a field, so direct methods (inverses, etc.) can be more complicated.

2. **Factor C into Prime Powers**
   - Express C as: $C = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$. - Example: If $C = 12$, then $12 = 2^2 \times 3^1$.

3. **Compute $F(n)$ Mod Each Prime Power**
   - For each prime power $p_i^{a_i}$, compute: $F(n) \bmod p_i^{a_i}$. - Techniques for prime powers often involve:
   - Euler's theorem (generalized).
   - Hensel's lemma (for certain polynomial lifts).
   - Lifting exponent lemmas (for factorials, binomial coefficients, etc.).

4. **Use the Chinese Remainder Theorem (CRT)**
   - The prime-power factors $p_i^{a_i}$ are pairwise coprime.
   - By CRT, if you know:

   $$x \equiv r_i \pmod{p_i^{a_i}}$$

   for each i, then there is a **unique** solution for

   $$x \bmod \left(p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}\right),$$

   which is $x \bmod C$.

5. **Example:** $C = 12$

6. Factor 12 into prime powers: $12 = 2^2 \times 3^1$.
7. Compute:

$$F(n) \bmod 4, \quad F(n) \bmod 3.$$

8. Combine the results using CRT:
   - Suppose $F(n) \equiv r_4 \pmod 4$ and $F(n) \equiv r_3 \pmod 3$.
   - There is a unique r_{12} with

$$r_{12} \equiv r_4 \pmod 4, \quad r_{12} \equiv r_3 \pmod 3.$$

   - Then $r_{12} \equiv F(n) \pmod{12}$.

Chinese Remainder Theorem (CRT) Usage for Avoiding Overflow

- We need to compute a function $F(0)$, which can fit in a 32-bit integer.
- However, intermediate calculations may cause overflow if performed directly.

Approach: Using Modular Arithmetic 1. **Select a Large Modulus** M
- Choose M as a product of several prime numbers:

$$M = p_1 \times p_2 \times \cdots \times p_k$$

- Ensure that M is large enough (e.g., more than 32 bits).
- If $F(0) < M$, then computing $F(0) \mod M$ is equivalent to computing $F(0)$.

2. **Break Down the Computation Using Primes**
   - Instead of computing $F(0) \mod M$ directly, compute:

$$F(0) \mod p_1, \quad F(0) \mod p_2, \quad \ldots, \quad F(0) \mod p_k$$

   - Each prime p_i is small enough to avoid overflow.

3. **Example of Prime Factorization** - Suppose we choose:

$$M = 257 \times 263 \times 269 \times 271$$

   - Each prime p_i is small enough for safe modular computations.

4. **Reconstruct the Result Using CRT**
   - Once we have $F(0) \mod p_i$ for all p_i, apply the **Chinese Remainder Theorem (CRT)** to reconstruct $F(0) \mod M$. - Since the primes are pairwise coprime, CRT guarantees a unique solution.

Others

- The Chicken McNugget Theorem states that for any two relatively prime positive integers m and n, the greatest integer that $\mathbf{cannot}$ be written in the form $am + bn$ for $\mathbf{nonnegative}$ integers a and b is $mn - m - n$.
- A consequence of the theorem is that there are exactly $\frac{(m-1)(n-1)}{2}$ positive integers which cannot be expressed in the form $am + bn$.
- The Generalized form of the Chicken McNugget Theorem states that for any two positive integers m and n, all multiples of $\gcd(m, n)$ greater than $\operatorname{lcm}(m, n) - m - n$ are representable in the form $am + bn$ for some positive integers a, b.

Arithmetic series

All three take a **limit** x, not a count — mixing the two conventions is the easy mistake here.

Sum	Formula
$1 + 2 + \cdots + x$	$\dfrac{x(x+1)}{2}$
odd numbers $\le x$	$\left\lfloor \dfrac{x+1}{2} \right\rfloor^2$
even numbers $\le x$	$\left\lfloor \dfrac{x}{2} \right\rfloor \left(\left\lfloor \dfrac{x}{2} \right\rfloor + 1 \right)$

The sum of the **first** k even numbers is $k(k+1)$ — that is a different question from the sum of even numbers up to a limit, and the two agree only when $x = 2k$.

For $\log_b a$ on integers, do not use `log(a) / log(b)`: it is off by one near exact powers. Loop, or compute a candidate and correct it by comparing $b^{\text{candidate}}$ against a.

Combinatorics

Properties

Binomial coefficients have many different properties. Here are the simplest of them:

- Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

- Sum over k:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n$$

- Sum over n:

$$\sum_{m=0}^{n} \binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k:

$$\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

$$\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n2^{n-1}$$

- Connection with the **Fibonacci numbers**:

$$\binom{n}{0} + \binom{n-1}{1} + \cdots + \binom{n-k}{k} + \cdots + \binom{0}{n} = F_{n+1}$$

Counting Integer Compositions with Upper Bounds

Suppose we want the number of ordered n-tuples

$$(x_1, x_2, \ldots, x_n)$$

of nonnegative integers satisfying

$$x_1 + x_2 + \cdots + x_n = S, \quad 0 \le x_i \le U (\forall i).$$

Denote this count by

$$F(n, S; U).$$

---

1. The "Unbounded" Baseline

If there were **no upper bound** on each x_i (only $x_i \ge 0$), then the number of solutions to

$$x_1 + \cdots + x_n = S, \quad x_i \ge 0$$

is given by the stars-and-bars formula:

$$G(n, S) = \binom{n + S - 1}{S}.$$

---

2. Imposing $x_i \le U$ via Inclusion–Exclusion

We want to exclude any solution in which some $x_i > U$. Define

$$A_i = \{x : x_i \ge U + 1\}.$$

By inclusion–exclusion,

$$F(n, S; U) = |\{x : \sum x_i = S, x_i \ge 0\}| - |A_1 \cup \cdots \cup A_n|.$$

In expanded form:

$$F(n, S; U) = \sum_{r=0}^{n} (-1)^r \sum_{1 \le i_1 < \cdots < i_r \le n} |A_{i_1} \cap \cdots \cap A_{i_r}|.$$

If r specific indices are forced to satisfy $x_{i_k} \ge U + 1$, shift each of those by $(U + 1)$. The total sum then becomes $S - r(U + 1)$, distributed freely among all n variables:

$$|A_{i_1} \cap \cdots \cap A_{i_r}| = G(n, S - r(U + 1)) = \binom{n + (S - r(U + 1)) - 1}{S - r(U + 1)},$$

provided $S - r(U+1) \geq 0$. Summing over all choices of r variables ($\binom{n}{r}$ ways) yields the final closed-form.

---

3. Final Formula

$$\boxed{F(n,S;U) = \sum_{r=0}^{\lfloor S/(U+1)\rfloor} (-1)^r \binom{n}{r} \binom{n + (S - r(U+1)) - 1}{S - r(U+1)}.}$$

- If $S < 0$ or $S > nU$, then $F(n,S;U) = 0$.
- Otherwise, let $r_{\max} = \lfloor S/(U+1) \rfloor$.

---

4. Special Cases

1. **Unrestricted ($U = \infty$)**: Then $(U+1) > S$, so only $r = 0$ survives. We recover
$$F(n,S;\infty) = \binom{n+S-1}{S}.$$
2. **Binary Variables ($U = 1$)**: Each $x_i \in \{0,1\}$. We get
$$F(n,S;1) = \sum_{r=0}^{\lfloor S/2 \rfloor} (-1)^r \binom{n}{r} \binom{n + (S-2r) - 1}{S-2r}.$$
A direct combinatorial argument shows this equals $\binom{n}{S}$.
3. **Ternary Bound ($U = 2$)**: Each $x_i \in \{0,1,2\}$. Then
$$F(n,S;2) = \sum_{r=0}^{\lfloor S/3 \rfloor} (-1)^r \binom{n}{r} \binom{n + (S-3r) - 1}{S-3r}.$$
This was exactly the core subproblem when counting "7/8/9 slices summing to K."

---

5. Efficient Implementation

When n and S can be as large as 10^5, we:

1. Precompute factorials $\mathrm{fact}[i] = i! \bmod M$ for $i = 0 \ldots N_{\max}$,
2. Precompute inverse-factorials $\mathrm{invFact}[i] = (i!)^{-1} \bmod M$ via
$$(i!)^{-1} = (i!)^{M-2} \bmod M, \quad M = 10^9 + 7,$$

using fast exponentiation. 3. Then

$$\binom{n}{r} = \text{fact}[n] \times \text{invFact}[r] \times \text{invFact}[n-r] \bmod M,$$

in $O(1)$ time per query.

Putting it all together:

```
const int MOD = 1000000007;
static const int MAXN = 200000; // big enough for n + S shifts

long long fact[MAXN+1], invFact[MAXN+1];

// (1) Fast exponentiation to compute a^p % MOD
long long modexp(long long a, long long p) {
  long long res = 1;
  while(p > 0) {
    if (p & 1) res = (res * a) % MOD;
    a = (a * a) % MOD;
    p >>= 1;
  }
  return res;
}

// (2) Precompute factorials and inverse factorials
void initFactorials() {
  fact[0] = 1;
  for(int i = 1; i <= MAXN; i++) {
    fact[i] = (fact[i-1] * i) % MOD;
  }
  invFact[MAXN] = modexp(fact[MAXN], MOD - 2);
  for(int i = MAXN; i >= 1; i--) {
    invFact[i-1] = (invFact[i] * i) % MOD;
  }
}

// (3) Binomial coefficient nCr % MOD
long long nCr(int n, int r) {
  if (r < 0 || r > n) return 0;
  return ((fact[n] * invFact[r]) % MOD * invFact[n-r]) % MOD;
}

// (4) Inclusion-Exclusion formula for 0 ≤ xi ≤ U, sum = S
long long countBounded(int n, int S, int U) {
  // If S out of [0, nU], no solutions
  if (S < 0 || S > 1LL * n * U) return 0;
  int rmax = S / (U + 1);
  long long ans = 0;
  for(int r = 0; r <= rmax; r++) {
    // Choose which r variables exceed U
    long long choose = nCr(n, r);
    int rem = S - r * (U + 1);
```

```
    // Distribute rem among n without bound:
    long long ways = nCr( n + rem - 1, rem );
    long long term = (choose * ways) % MOD;
    if (r & 1) {
      term = (MOD - term) % MOD;  // subtract if r is odd
    }
    ans = (ans + term) % MOD;
  }
  return ans;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  initFactorials();

  int T;
  cin >> T;
  while(T--) {
    int n, S, U;
    cin >> n >> S >> U;
    cout << countBounded(n, S, U) << "\n";
  }
  return 0;
}
```

- `initFactorials()` populates `fact[i]` and `invFact[i]` up to `MAXN`.
- `nCr(n,r)` returns $\binom{n}{r} \bmod 10^9+7$.
- `countBounded(n,S,U)` implements

$$\sum_{r=0}^{\lfloor S/(U+1)\rfloor} (-1)^r \binom{n}{r} \binom{n + (S - r(U+1)) - 1}{S - r(U+1)}.$$

---

6. Key Takeaways

- **Stars & Bars** handles $x_i \geq 0$ with $\sum x_i = S \to \binom{n+S-1}{S}$.
- To enforce $x_i \leq U$, use **inclusion–exclusion**, subtracting solutions where any $x_i \geq U+1$.
- The final formula is

$$F(n, S; U) = \sum_{r=0}^{\lfloor S/(U+1)\rfloor} (-1)^r \binom{n}{r} \binom{n + (S - r(U+1)) - 1}{S - r(U+1)}.$$

- Precompute factorials and inverse factorials modulo $10^9 + 7$ to answer each binomial in $O(1)$.

Divisability Rules

2

A number is divisible by 2 if its last digit is even (0, 2, 4, 6, or 8).

3

A number is divisible by 3 if the sum of its digits is divisible by 3.

4

A number is divisible by 4 if the number formed by its last two digits is divisible by (e.g., 12, 16, 552).

5

A number is divisible by 5 if its last digit is 0 or 5.

6

A number is divisible by 6 if it is divisible by both 2 and 3.

7

A number is divisible by 7 if you double the last digit, subtract it from the rest of the number, and the result is divisible by 7. Repeat this process until you get a small enough number to check directly.

`672 (Double 2 is 4, 67−4=63, and 63÷7=9) Yes`

8

A number is divisible by 8 if the number formed by its last three digits is divisible by 8.

9

A number is divisible by 9 if the sum of its digits is divisible by 9.

10

A number is divisible by 10 if its last digit is 0.

11

A number is divisible by 11 if the difference between the sum of its digits in odd positions and the sum of its digits in even positions is divisible by 11.

12

A number is divisible by 12 if it is divisible by both 3 and 4.

Properties of Bitwise Operations

- $a|b = a \oplus b + a\&b$
- $a \oplus (a\&b) = (a|b) \oplus b$
- $b \oplus (a\&b) = (a|b) \oplus a$
- $(a\&b) \oplus (a|b) = a \oplus b$

Addition:

- $a + b = a|b + a\&b$
- $a + b = a \oplus b + 2(a\&b)$

Subtraction:

- $a - b = (a \oplus (a\&b)) - ((a|b) \oplus a)$
- $a - b = ((a|b) \oplus b) - ((a|b) \oplus a)$
- $a - b = (a \oplus (a\&b)) - (b \oplus (a\&b))$
- $a - b = ((a|b) \oplus b) - (b \oplus (a\&b))$

Built-in Bitwise Functions in C++

- `__builtin_popcount(n)` : Returns the number of set bits in `n` .
- `__builtin_popcountll(n)` : Returns the number of set bits in `n` (for `long long`).
- `__builtin_ctz(n)` : Returns the number of trailing zeros in `n` .
- `__builtin_ctzll(n)` : Returns the number of trailing zeros in `n` (for `long long`).
- `__builtin_clz(n)` : Returns the number of leading zeros in `n` .
- `__builtin_clzll(n)` : Returns the number of leading zeros in `n` (for `long long`).
- `__builtin_ffs(n)` : Returns the index of the least significant set bit in `n` .
- `__builtin_ffsll(n)` : Returns the index of the least significant set bit in `n` (for `long long`).
- `__builtin_parity(n)` : Returns the parity of `n` (number of set bits modulo 2).
- `__builtin_parityll(n)` : Returns the parity of `n` (number of set bits modulo 2) (for `long long`).

Alternative Representations:

- $a + b = a \oplus b + 2(a \& b)$
- $a + b = 2(a | b) - (a \oplus b)$

Single-bit idioms

Want	Write
set bit `i`	`n | (1LL << i)`
clear bit `i`	`n & ~(1LL << i)`
toggle bit `i`	`n ^ (1LL << i)`
test bit `i`	`(n >> i) & 1`
clear the lowest set bit	`n & (n - 1)`
isolate the lowest set bit	`n & -n`

Always `1LL << i`, never `1 << i`. `1` is an `int`, so `1 << i` is undefined behaviour for `i >= 31` even when you assign the result to a `long long`. This is the single most common bit-manipulation bug and it does not warn.

Prefer `(n >> i) & 1` to `n & (1 << i)` when you want a boolean: the second returns the bit's *value*, so it is `1 << i` rather than `1` when set, which breaks `==` comparisons.

For counting bits use the built-ins listed above rather than hand-rolled macros; note `__builtin_popcount` takes an `unsigned int`, so a `long long` needs the `ll` suffix or the high half is silently dropped.

Geometry

Triangle

- **Area of a Triangle Given Three Vertices**
- Given three points in a 2D plane:
  $(0, 0), (X, Y), (A, B)$
- The formula for the area of a triangle given three vertices (x_1, y_1), (x_2, y_2), and (x_3, y_3) is:
  $\text{Area} = \frac{1}{2}|x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$
- Substituting $(0, 0)$, (X, Y), and (A, B):
  $\text{Area} = \frac{1}{2}|0(Y - B) + X(B - 0) + A(0 - Y)|$
- Simplifying the equation:
  $\text{Area} = \frac{1}{2}|XB - YA|$

`complex<double>` as a point

`Geometry.cpp` and `Circle.cpp` use `complex<double>` for points, so the standard library gives you most vector algebra for free. What each function does, on `c = (4, 6)`:

Call	Result	Meaning
`real(c)` / `imag(c)`	`4` / `6`	the x and y components
`a + b`, `a - b`		vector addition and subtraction
`abs(c)`	`7.2111`	length, `sqrt(norm(c))` — so `abs(a - b)` is the distance
`norm(c)`	`52`	length **squared**, `x*x + y*y` — *not* the length
`arg(c)`	`0.876058`	the angle from the +x axis, `atan2(y, x)`
`conj(c)`	`4 - 6i`	reflection across the x axis
`polar(r, t)`		the vector of length `r` at angle `t`
`a * polar(1.0, t)`		`a` rotated counter-clockwise by `t`
`(conj(a) * b).real()`		the dot product of `a` and `b`
`(conj(a) * b).imag()`		the cross product of `a` and `b`

Two things to watch. `norm` is the squared length, which is the most common misread here. And the standard only specifies `complex` for `float`, `double` and `long double` — there is no `complex<long long>`, which is

why exact integer geometry lives in `Convex_Hull.cpp` and `Polygon.cpp` on a plain struct instead.

Applications of FFT

1. Ploynomials Multiplication

▶ statement

2. Ploynomials power

   ▶ statement

3. Multiply large numbers

▶ statement

4. Generate all pair sum
   ▶ statement

▶ solution

5. Generate all pair difference $ai - aj$
   ▶ statement

▶ solution

6. Generate all subarray sum
   ▶ statement

▶ solution

7. Generate all subset sum
   ▶ statement

▶ solution

FFT relative to index

Some ideas to use the index as the exponent in the polynomial and the element as the coefficient

1. Multiplication with distance
   ▶ statement

▶ solution

2. same as above but different representation
   ▶ statement

▶ solution

3. Problem 3
   ▶ statement

▶ solution

String matching

1. Match a pattern consist of a wildcard character
   ▶ statement

▶ solution

Multiplication with arbitrary modulus

Here we want to achieve the same goal as in previous section. Multiplying two polynomial $A(x)$ and $B(x)$, and computing the coefficients modulo some number M. The number theoretic transform only works for certain prime numbers. What about the case when the modulus is not of the desired form?

One option would be to perform multiple number theoretic transforms with different prime numbers of the form $c2^k + 1$, then apply the Chinese Remainder Theorem to compute the final coefficients.

Another options is to distribute the polynomials $A(x)$ and $B(x)$ into two smaller polynomials each

$$A(x) = A_1(x) + A_2(x) \cdot C$$
$$B(x) = B_1(x) + B_2(x) \cdot C$$

with $C \approx \sqrt{M}$.

Then the product of $A(x)$ and $B(x)$ can then be represented as:

$$A(x) \cdot B(x) = A_1(x) \cdot B_1(x) + (A_1(x) \cdot B_2(x) + A_2(x) \cdot B_1(x)) \cdot C + (A_2(x) \cdot B_2(x)) \cdot C^2$$

$A_1(x)$ -> is the polynomial containing the remiander of the coefficients of $A(x)$ when divided by C.

$A_2(x)$ -> is the polynomial containing the coefficients of $A(x)$ divided by C.

$B_1(x)$ -> is the polynomial containing the remiander of the coefficients of $B(x)$ when divided by C.

$B_2(x)$ -> is the polynomial containing the coefficients of $B(x)$ divided by C.

The polynomials $A_1(x)$, $A_2(x)$, $B_1(x)$ and $B_2(x)$ contain only coefficients smaller than $\sqrt{M}$, therefore the coefficients of all the appearing products are smaller than $M \cdot n$, which is usually small enough to handle with typical floating point types.

This approach therefore requires computing the products of polynomials with smaller coefficients (by using the normal FFT and inverse FFT), and then the original product can be restored using modular addition and multiplication in $O(n)$ time.

Pushing NTT Beyond Limits: Fast Half-NTT for Huge Polynomial Multiplication

Motivation: The Limits of NTT

NTT (Number Theoretic Transform) is a powerful tool for multiplying polynomials efficiently under a modulus, with time complexity $O(n \log n)$. It requires a special kind of prime modulus:

$$\text{mod} = k \cdot 2^s + 1$$

A popular one is:

$$\boxed{998244353 = 119 \cdot 2^{23} + 1}$$

This prime allows NTT to work efficiently up to size $2^{23} = 8,388,608$, which is great — until it's not.

What if you're given two large polynomials with degrees:

- $n = 2^{24}$
- $m = 2^{24}$

Then the convolution size becomes:

$$n + m - 1 \approx 2^{25}$$

This **exceeds the NTT limit** for `998244353`, making standard NTT **inapplicable**.

The Problem

We want to perform polynomial multiplication under modulus `998244353`, but input size is larger than 2^{23}, the maximum NTT-friendly length.

You **can't**:

- Use normal NTT — the root of unity for such size doesn't exist under this modulus.

You **don't want**:

- Floating-point FFT (inaccurate for exact integer modulo arithmetic)
- CRT with multiple moduli (extra complexity and memory)

The Solution: Recursive Fast Half-NTT

Key Idea

Instead of building a massive NTT transform, we:

- **Recursively split** the problem into two subproblems using root of unity identities.
- **Solve smaller convolutions** (within size limit).
- **Combine** them using inverse interpolation.

This method:

- Never requires an NTT of size > 2^{23}
- Keeps all operations within `998244353`
- Works efficiently even for huge inputs

How It Works

Step 1: Root-Based Decomposition

Given a polynomial $A(x)$ of degree n, split it like this:

```
A_minus[i] = A[i] + w^x * A[i + n/2]
A_plus[i]  = A[i] - w^x * A[i + n/2]
```

Where:

- ω is a primitive root of unity modulo `998244353`
- x determines which power of root to use at each level

Do the same for $B(x)$

Step 2: Recursive Multiplication

Now multiply:

- $A_- \cdot B_-$ using `x/2`
- $A_+ \cdot B_+$ using `x/2 + mod/2`

Repeat this recursively until size ≤ threshold (`lim` , e.g. 64), then use naive $O(n^2)$ multiplication.

Step 3: Interpolate

After computing the two products, combine them back:

```
C[i]          = (res_minus[i] + res_plus[i]) / 2
C[i + n/2]    = (res_minus[i] - res_plus[i]) / (2 * sqrt_c)
```

All operations are done modulo `998244353` .

Code Skeleton

```
vector<int> fast_ntt_poly_mul(vector<int>& a, vector<int>& b) {
    // Resize a, b to next power of two above n + m - 1
    // Recursively multiply
    // Return trimmed result
}
```

This is a clean and efficient recursive algorithm with minimal overhead.

Performance

- Handles sizes up to 2^{25} with **no problem**
- Memory-efficient: uses divide-and-conquer strategy
- Competitive with iterative NTT for large inputs

When Should You Use This?

When:

- You have large polynomials (bigger than 2^{23})
- You want to stay within modulus `998244353`
- You don't want to deal with CRT

Avoid if:

- Your inputs are small (use iterative NTT instead)
- You don't have a good NTT-friendly modulus (e.g., $10^9 + 7$)

Can This Be Generalized?

Yes, **but only to other NTT-friendly primes**.

Requirements:

- The modulus $p = k \cdot 2^s + 1$
- Existence of roots of unity for various powers
- Ability to divide by 2 (modular inverse of 2 exists)

Examples:

- 998244353
- 167772161
- 7340033

Not applicable for:

- $10^9 + 7$, $10^9 + 9$, etc.
- Non-NTT-friendly primes

In such cases, you'll need multi-modulus + CRT or float FFT.

Summary

Feature	Recursive Fast-NTT
Max size	Any (adaptive)
Modulus	998244353 (or similar)
Time	$O(n \log n)$
Space	$O(n)$
Uses CRT?	No
Works with FFT?	No

Subarray Hashing

Problem Statement

Given two arrays `A` and `B` and `q` queries, each query consists of four integers l_i r_i L_i R_i such that $1 \leq l_i \leq r_i \leq n$ and $1 \leq L_i \leq R_i \leq n$. For each query, we need to check if the multiset of the subarray `A[l_i...r_i]` is equal to the multiset of the subarray `B[L_i...R_i]`.

Approach

Give each value five random numbers, and make five prefix sums over each value and only compare the sum of the subarrays.

Implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
inline int random(int l, int r){
    return rnd() % (r - l + 1) + l;
}

struct MultisetNode {
    ll a, b, c, d, e;
    MultisetNode() {
        a = random(1, 1e9);
        b = random(1, 1e9);
        c = random(1, 1e9);
        d = random(1, 1e9);
        e = random(1, 1e9);
    }
    MultisetNode (ll _a, ll _b, ll _c, ll _d, ll _e) : a(_a), b(_b), c(_c), d(_d), e(_e) {}
    MultisetNode operator+(const MultisetNode &other) const {
        return MultisetNode(a + other.a, b + other.b, c + other.c, d + other.d, e + other.e);
    }
    MultisetNode operator-(const MultisetNode &other) const {
        return MultisetNode(a - other.a, b - other.b, c - other.c, d - other.d, e - other.e);
    }
    bool operator==(const MultisetNode &other) const {
        return a == other.a && b == other.b && c == other.c && d == other.d && e == other.e;
    }
};
void solve(){
    int n, q;
    cin >> n >> q;
    vector<MultisetNode> A(n + 1), B(n + 1);
    A[0] = B[0] = MultisetNode(0, 0, 0, 0, 0);
    map<ll, MultisetNode> mp;
    for(int i = 1; i <= n; i++){
        ll x;
        cin >> x;
        if(mp.find(x) == mp.end()){
            mp[x] = MultisetNode();
        }
        A[i] = A[i - 1] + mp[x];
    }
    for(int i = 1; i <= n; i++){
        ll x;
```

```
        cin >> x;
        if(mp.find(x) == mp.end()){
            mp[x] = MultisetNode();
        }
        B[i] = B[i - 1] + mp[x];
    }
    while(q--){
        int l, r, L, R;
        cin >> l >> r >> L >> R;
        if(A[r] - A[l - 1] == B[R] - B[L - 1]){
            cout << "YES\n";
        } else {
            cout << "NO\n";
        }
    }
}
```

C++17 Lambdas

- `[]` No capture | `[&]` By reference | `[=]` By value

```
// 1. Non-Recursive

auto add = [](int a, int b) { return a + b; };
add(5, 3);
```

```
// 2. Recursive (auto&& | function<r_type(param,param)>)

auto gcd = [&](int a, int b, auto&& gcd) -> int {
    return b == 0 ? a : gcd(b, a % b, gcd);
};
gcd(20, 30, gcd);
```